

Plan de desarrollo

Sistema de Registro y Monitoreo de Generación Solar por Departamento Instrucciones de trabajo para Claude Code · Competencia de Programación con IA, 11–12 de septiembre de 2026

Este documento fija lo que se construye y en qué orden. Las decisiones de la sección 0 y el esquema de la sección 2 no se reabren: si algo del plan no se puede cumplir tal cual, se reporta y se propone la alternativa antes de cambiarlo.

Desviaciones registradas durante la ejecución: base de datos MySQL 8.4 en producción en lugar de PostgreSQL (costo del recurso en Laravel Cloud); mosaicos de Esri en lugar de OpenStreetMap/CARTO (ver `DISEÑO.md`, sección 15); frontend público en vistas Blade propias en lugar de widgets de Filament, para poder cumplir el sistema de diseño. Todas se documentaron en `USO-DE-IA.md`.

0. Decisiones tomadas

Pieza	Decisión	Por qué
Framework	Laravel 13 (PHP 8.3+)	Obligatorio por bases
Panel admin / CRUD	Filament	Cubre RF-01→RF-09 y RF-17 casi completos
Base de datos	PostgreSQL (SQLite en local)	Laravel Cloud y Railway lo provisionan
Mapa	Leaflet con mosaicos sin clave de API	Sin tarjeta, sin facturación
Gráficas	Chart.js	Ligero, sin dependencias de servidor
API	Laravel <code>apiResource</code> + API Resources	Nativo
Doc API	<code>dedoc/scramble</code> (OpenAPI automático)	Genera la documentación leyendo los controladores
Deploy	Laravel Cloud (plan B: Railway)	Deploy nativo de Laravel con base de datos incluida
Auth	Panel autenticado; tablero público sin login	Evita fricción al evaluar

Si Filament 5 da problemas de compatibilidad al instalar, usar la mayor anterior sin más discusión.

Dos prioridades transversales:

- Deploy temprano.** La URL pública debe existir antes de escribir lógica de negocio; después, cada fase termina con `git push` y despliegue automático.
- Datos de demostración con historia.** Alimentan a la vez el tablero, el mapa, las alertas y la proyección; un seeder pobre arruina los cuatro. Se les dedica una fase propia con parámetros exactos.

1. Esquema de base de datos

Es lo único que la IA no diseña: se implementa tal cual. Nombres en español.

```
departamentos
  id
  nombre
  codigo          string(3) unique      // GT-IZA, GT-GUA...
  cabecera        string nullable
  latitud          decimal(10,7)        // centroide, para centrar el mapa
  longitud         decimal(10,7)
  timestamps

modelos_panel          // RF-02
  id
  marca                string
  modelo               string
  potencia_kw          decimal(8,3)      // potencia nominal por unidad
  eficiencia            decimal(5,2) nullable
  estado               enum(activo, inactivo, descontinuado) default activo
  timestamps
  unique(marca, modelo)

granjas                // RF-03, RF-04, RF-07, RF-09
  id
  nombre               string
  departamento_id      FK -> departamentos
  municipio            string nullable
  direccion            string nullable
  latitud              decimal(10,7)
  longitud              decimal(10,7)
  familias_beneficiadas integer default 0
  generacion_esperada_mensual_kwh decimal(14,2) default 0
  fecha_instalacion    date nullable
  activa               boolean default true // desactivar, no borrar
  timestamps
  softDeletes

granja_panel           // RF-05 (pivot con datos)
  id
  granja_id            FK -> granjas (cascade)
  modelo_panel_id      FK -> modelos_panel
  cantidad             unsignedInteger
  fecha_instalacion    date nullable
  timestamps
  unique(granja_id, modelo_panel_id)

generaciones           // RF-08, RF-09
  id
  granja_id            FK -> granjas (cascade)
  periodo              date              // siempre día 1 del mes
  generacion_real_kwh   decimal(14,2)
  generacion_esperada_kwh decimal(14,2) // snapshot del período
  timestamps
  unique(granja_id, periodo)
  index(periodo)

alertas                // RF-14
```

```

id
granja_id          FK -> granjas
generacion_id      FK -> generaciones (cascade)
periodo            date
generacion_esperada_kwh decimal(14,2)
generacion_real_kwh decimal(14,2)
porcentaje_desviacion decimal(6,2) // negativo = por debajo
estado             enum(activa, revisada, resuelta) default activa
timestamps
index(estado)

```

Reglas de cálculo (van en `config/solar.php` , nunca escritas a mano en otro archivo):

```

return [
    'factor_co2_kg_por_kwh' => 0.40,          // RF-10
    'umbral_alerta'         => 0.80,          // RF-14: real <= 80% de esperada
    'meses_historicos_proyeccion' => 12,
];

```

Valores derivados — atributos calculados, nunca columnas:

- `capacidad_instalada_kw` = `SUM(granja_panel.cantidad × modelos_panel.potencia_kw)` (RF-06)
- `co2_evitado_kg` = `generacion_acumulada_kwh × 0.40` (RF-10)
- `porcentaje_desviacion` = `(real - esperada) / esperada × 100`

Guardarlos como columnas introduce riesgo de desincronización cada vez que cambian los paneles de una granja; por eso se resuelven con accessors o scopes con subconsultas.

2. Archivo `CLAUDE.md`

Se crea en la raíz antes de escribir código y se respeta en todas las fases:

```

# Proyecto: Sistema de Generación Solar Guatemala

Laravel 13 + Filament + PostgreSQL + Leaflet.

## Reglas no negociables
- Idioma del dominio: ESPAÑOL. Tablas, modelos, campos y rutas en español (granjas, modelos_panel, generaciones, alertas). Código y comentarios en español.
- Nunca inventes API de Filament de memoria. Si no estás seguro de una firma, consultá la documentación oficial o revisá un archivo generado por el propio comando `make:` antes de escribir.
- Valores derivados = accessors o scopes. Nunca columnas duplicadas en BD.
- Constantes de negocio siempre desde config('solar.*'). Cero números mágicos.
- Ningún secreto en el código. Todo va a .env y .env.example.
- Después de cada cambio de migración: ejecutá `php artisan migrate:fresh --seed` y confirmá que corre sin error antes de decir que terminaste.
- No refactorices archivos que no te pedí tocar.
- Commits pequeños, mensaje en español, formato: `feat: ...` / `fix: ...`

## Antes de terminar una tarea
1. Ejecutá el código, no asumas que sirve.
2. Reportá qué probaste y qué salida obtuviste.

```

3. Si algo quedó a medias, decílo explícitamente. No lo escondas.

Contexto de negocio

- 22 departamentos de Guatemala.
- CO2 evitado = kWh × 0.40 kg.
- Alerta cuando generación real ≤ 80% de la esperada en el mismo período.
- Períodos son mensuales, almacenados como fecha del día 1 del mes.

3. Fases

Cada fase se ejecuta en un contexto limpio, con un objetivo, y termina con un commit y un push. Antes de escribir código en cada fase, proponer el plan de cambios y esperar confirmación. Al terminar, reportar qué se ejecutó y qué salida se obtuvo.

FASE 0 — Andamiaje y deploy vacío

Objetivo: URL pública viva antes de escribir lógica.

```
composer create-project laravel/laravel generacion-solar
cd generacion-solar
composer require filament/filament
php artisan filament:install --panels
composer require dedoc/scramble
git init && git add . && git commit -m "chore: proyecto inicial"
# crear repo en GitHub y push
```

Luego, en Laravel Cloud (o Railway):

1. Conectar el repositorio de GitHub.
2. Provisionar la base de datos (las variables se inyectan solas).
3. Configurar el comando de despliegue: `php artisan migrate --force`.
4. Verificar que la URL carga.

No se pasa a la Fase 1 hasta que la URL pública responda.

FASE 1 — Modelo de datos

Creá las migraciones, modelos Eloquent y factories para este esquema exacto. No agregues campos que no estén listados ni cambies nombres. [esquema de la sección 1]

Además: - `config/solar.php` con `factor_co2_kg_por_kwh = 0.40`, `umbral_alerta = 0.80`, `meses_historicos_proyeccion = 12`. - Relaciones Eloquent en ambos sentidos, con tipado de retorno. - Casts apropiados (decimal, date, enum). - En el modelo Granja: accessor `capacidadInstaladaKw` que suma `cantidad × potencia_kw` de sus paneles, y accessor `co2EvitadoKg`. - Seeder `DepartamentoSeeder` con los 22 departamentos de Guatemala reales, con su cabecera y coordenadas del centroide. - Corré `php artisan migrate:fresh --seed` y mostrame la salida.

Criterio de aceptación: en `php artisan tinker`, los 22 departamentos existen y todas las coordenadas caen dentro de Guatemala (lat 13.7–17.8, lon -92.2 a -88.2). Reportar la comprobación.

FASE 2 — CRUDs en Filament

Generá los recursos de Filament para `ModeloPanel`, `Granja` y `Generacion`.

- **ModeloPanel:** formulario con marca, modelo, potencia_kw, eficiencia, estado. Tabla con búsqueda por marca/modelo y filtro por estado.
- **Granja:** formulario con nombre, departamento (select con búsqueda), municipio, latitud, longitud, familias_beneficiadas, generacion_esperada_mensual_kwh, fecha_instalacion, activa. Tabla con columnas de departamento, capacidad instalada calculada, familias, y filtro por departamento y por estado activo. Acción de desactivar (no borrar).
- **Relation manager** en Granja para administrar sus paneles (modelo de panel + cantidad), mostrando la capacidad total resultante.
- **Generacion:** formulario con granja, periodo (selector de mes, se guarda como día 1), generacion_real_kwh, generacion_esperada_kwh. Al elegir granja, precargar la esperada desde generacion_esperada_mensual_kwh.

Validaciones (RF-17): campos obligatorios, numéricos no negativos, latitud entre -90 y 90, longitud entre -180 y 180, período único por granja, cantidad de paneles ≥ 1 . Mensajes de error en español.

Criterio de aceptación: crear una granja con dos modelos de panel desde el panel y ver la capacidad total calculada en su ficha.

FASE 3 — Alertas

Implementá la regla de alertas (RF-14):

- `GeneracionObserver` que en `saved` evalúa: si `generacion_real_kwh <= generacion_esperada_kwh * config('solar.umbra1_alerta')`, crea o actualiza la Alerta correspondiente a esa generación; si deja de cumplirse, elimina la alerta existente.
- Guardá esperada, real y porcentaje_desviacion en la alerta.
- Recurso Filament de solo lectura para Alertas: granja, departamento, período, esperada, real, % de desviación con badge de color (rojo $< -40\%$, ámbar entre -20% y -40%), filtro por estado y por departamento.
- Comando `artisan solar:evaluar-alertas` que recorre todas las generaciones y reconstruye las alertas, para poder regenerarlas tras cargar datos masivos.
- Escribí un test que confirme: 79% dispara alerta, 80% dispara alerta (el requisito dice "al menos 20% por debajo", o sea $\leq 80\%$ es alerta), 81% no dispara.

Criterio de aceptación: los tres casos del test pasan. El caso borde del 80 % exacto queda documentado en el README.

FASE 4 — Tablero y reportes

Construí el tablero con estos componentes, todos alimentados por consultas agregadas (nada de cargar colecciones completas en memoria):

Stats nacionales: total de granjas, total de paneles instalados, capacidad instalada total en kW, generación acumulada en kWh, total de familias beneficiadas, CO2 evitado en kg y en toneladas.

Gráfica de barras: ranking de departamentos por generación acumulada (top 10).

Gráfica de líneas: generación real vs esperada por mes, a nivel nacional, últimos 12 meses.

Tabla de reporte por departamento (RF-12): una fila por departamento con cantidad de granjas, cantidad de paneles, capacidad instalada kW, generación acumulada kWh, familias beneficiadas y CO2 evitado kg. Ordenable por cualquier columna. Con filtro de rango de fechas que afecte las columnas de generación.

Widget de alertas activas: las 10 más recientes con enlace a la granja.

Todos los números formateados con separador de miles y las unidades visibles.

Criterio de aceptación: las cifras del tablero coinciden con las que devuelve una consulta SQL directa sobre los datos cargados.

FASE 5 — Mapa interactivo

Creá una página `/mapa` (pública, sin login) con Leaflet y mosaicos sin clave de API:

- Centrada en Guatemala, zoom inicial que muestre todo el país.
- Un marcador por cada granja activa, con tamaño según capacidad instalada y color según generación acumulada, siguiendo la escala de `DISEÑO.md`.
- Popup al hacer clic: nombre de la granja, departamento, municipio, capacidad instalada kW, generación acumulada kWh, familias beneficiadas, CO2 evitado kg, y distintivo si tiene alerta activa.
- Filtro lateral por departamento que oculta/muestra marcadores sin recargar.
- Leyenda.
- Los datos se consumen desde `/api/v1/granjas/mapa`, no embebidos en el Blade.
- Responsive: en móvil el filtro se apila sobre el mapa.

Criterio de aceptación: el mapa es parte de la rúbrica ("mapa interactivo y experiencia de usuario"), así que se revisa contra `DISEÑO.md`: tipografía, jerarquía del popup, estados hover. Nada con el aspecto por defecto de Leaflet.

FASE 6 — Proyección

Método elegido: regresión lineal por mínimos cuadrados sobre los últimos 12 períodos.

Justificación:

Los datos de generación mensual tienen tendencia y ruido, con muy pocos puntos por granja. Una regresión lineal simple captura la tendencia sin sobreajustar, es determinista, no requiere entrenamiento ni dependencias externas, y cada coeficiente es explicable. Un modelo de ML con 12 puntos por granja sobreajustaría sin aportar precisión real. Cuando hay menos de 3 períodos, el sistema cae a promedio móvil, porque una recta con dos puntos no es una tendencia.

Creá `App\Services\ProyeccionService` con: - `proyectar(Granja $granja, int $meses = 3): array` — regresión lineal por mínimos cuadrados sobre los últimos `config('solar.meses_historicos_proyeccion')` períodos. Devuelve los meses proyectados con su valor, la pendiente, el intercepto y el R^2 . - Fallback a promedio móvil si hay menos de 3 períodos históricos; el resultado debe indicar qué método se usó. - Nunca devolver proyecciones negativas: piso en 0. - `compararProyeccionVsReal(Granja $granja)` — para períodos donde ya existe dato real, calcula qué habría proyectado el modelo con los datos previos y el error porcentual. - Widget en la ficha de granja: gráfica con histórico (línea sólida) + proyección a 3 meses (línea punteada), y una tabla de precisión histórica del modelo. - Tests con una serie perfectamente lineal (debe proyectar exacto) y con una serie plana.

Criterio de aceptación: los tests pasan y la ficha de una granja muestra los tres meses proyectados con la precisión histórica.

FASE 7 — API REST y documentación

Implementá la API REST versionada bajo `/api/v1` con API Resources:

- `GET /departamentos` y `GET /departamentos/{id}` (incluye estadísticas agregadas)
- `GET /granjas` (paginado, filtros: departamento_id, activa, búsqueda por nombre), `GET /granjas/{id}`
- `GET /granjas/mapa` — payload liviano para el mapa
- `GET /granjas/{id}/generaciones` (filtro por rango de períodos)
- `GET /granjas/{id}/proyeccion?meses=3`
- `GET /generaciones` (paginado, filtros por período y departamento)
- `GET /estadisticas/nacional` y `GET /estadisticas/departamentos`
- `GET /alertas` (filtro por estado y departamento)

Requisitos: respuestas con estructura consistente, códigos HTTP correctos, 404 con mensaje JSON legible, rate limiting, paginación con metadatos. Documentá todos los endpoints con anotaciones para Scramble y dejá la doc navegable en `/docs/api`.

Además generá `docs/API.md` con tabla de endpoints, parámetros y un ejemplo de respuesta por cada uno, por si la doc interactiva falla en producción.

Criterio de aceptación: cada endpoint probado con `curl`, incluidos los casos 404 y 422, con el código de estado y la forma de la respuesta reportados.

FASE 8 — Datos de demostración

Los parámetros son exactos; no se dejan a criterio del seeder.

Creá `DemoSeeder` que genere datos realistas: - 8 modelos de panel de marcas reales (Jinko, Trina, Canadian Solar, LONGi...) con potencias entre 0.35 y 0.7 kW. - 35 granjas distribuidas en al menos 15 de los 22 departamentos, con coordenadas reales dentro del departamento correspondiente (no aleatorias sobre el país). Nombres verosímiles tipo "Granja Solar Las Palmas", "Parque Solar Motagua". - Concentrá más granjas en departamentos de alta irradiación (Zacapa, El Progreso, Jutiapa, Santa Rosa, Izabal) para que el ranking cuente una historia. - Cada granja con 2 a 5 modelos de panel y cantidades entre 200 y 4000 unidades. - 18 meses de generación por granja, con: estacionalidad marcada (más generación en verano seco nov-abr, menos en época lluviosa may-oct), tendencia leve al alza, y ruido aleatorio de $\pm 8\%$. - **Exactamente 6 granjas deben quedar con bajo desempeño sostenido** (real entre 55% y 78% de la esperada en los últimos 3 meses) para que las alertas se vean pobladas. - 2 granjas con tendencia claramente creciente y 2 con tendencia decreciente, para que las proyecciones sean visualmente interesantes. - Familias beneficiadas proporcional a la capacidad instalada, entre 80 y 3500. - Al final corré `solar:evaluar-alertas` y mostrame cuántas alertas se generaron.

Criterio de aceptación: `migrate:fresh --seed` corre limpio en local y en producción; el tablero, el mapa, las alertas y las proyecciones se ven poblados.

FASE 9 — Pulido, documentación y presentación

- `README.md`: descripción, objetivos, stack, requisitos, instalación local paso a paso, comandos artisan propios, credenciales de acceso demo, URL pública, enlace al repositorio, modelo de datos.
- `docs/USO-DE-IA.md`: entregable obligatorio. Qué herramienta se usó, en qué fases, qué se revisó y corrigió manualmente, y qué decisiones tomó el equipo y no la IA (el esquema de datos, el método de proyección, el stack).
- Revisar que `.env` no esté en el repositorio y que `.env.example` sí.
- Verificar la aplicación en un viewport de 375 px: sin scroll horizontal, navegación usable.
- Presentación de 10–12 diapositivas con notas del orador.

Guion de la presentación:

1. Problema y alcance
 2. Arquitectura y modelo de datos (diagrama ER)
 3. Demo en vivo: CRUD → tablero → mapa → alerta → proyección
 4. Método de proyección y su justificación
 5. API REST y documentación
 6. Uso de IA: cómo se dirigió y qué controló el equipo
 7. Cierre: métricas del sistema
-

4. Reglas de trabajo para el agente

- Una fase por vez, un objetivo por fase. No combinar fases.
 - Proponer el plan de cambios antes de ejecutar; ejecutar solo tras confirmación.
 - Ejecutar y mostrar la salida. "Listo, ya funciona" sin salida no cuenta como evidencia.
 - No inventar sintaxis de Filament: verificar contra `vendor/filament` o un archivo generado por `make: .`
 - No tocar migraciones ya aplicadas en producción sin avisar.
 - Ante una instrucción de estilo, aplicar `DISEÑO.md` ; no improvisar colores, tipografía ni espaciados.
 - Si algo falla dos veces seguidas, detenerse y explicar la causa en lugar de intentar una tercera variante.
-

5. Lista de verificación contra los requerimientos

Se comprueba cada uno **en la URL de producción**, no en local:

- [x] RF-01 — 22 departamentos cargados y asociables
- [x] RF-02 — CRUD de modelos de panel con marca, modelo, potencia kW, estado
- [x] RF-03 — Crear, editar, consultar y **desactivar** granjas
- [x] RF-04 — Latitud y longitud almacenadas y visibles en el mapa
- [x] RF-05 — Paneles asociados a granja con cantidad
- [x] RF-06 — Capacidad instalada calculada automáticamente
- [x] RF-07 — Familias beneficiadas por granja
- [x] RF-08 — Generación real por período en kWh
- [x] RF-09 — Generación esperada comparable con la real
- [x] RF-10 — CO2 con factor 0.40 kg/kWh
- [x] RF-11 — Tablero con indicadores nacionales y por departamento
- [x] RF-12 — Reporte por departamento con las 6 métricas mínimas
- [x] RF-13 — Mapa con marcadores, selección e información de granja
- [x] RF-14 — Alerta cuando real $\leq$ 80% de esperada
- [x] RF-15 — Proyección implementada, documentada y justificada
- [x] RF-16 — API REST con departamentos, granjas, generación y estadísticas
- [x] RF-17 — Validaciones de obligatorios, rangos y relaciones

Entregables:

- [x] URL pública funcionando
- [x] Repositorio de GitHub accesible
- [x] Base de datos relacional
- [x] README con instalación y ejecución
- [x] Documentación de la API
- [x] Documento de uso de IA
- [x] Datos de demostración suficientes (tablero poblado, mapa lleno, alertas visibles, proyecciones con forma)
- [x] Presentación lista

